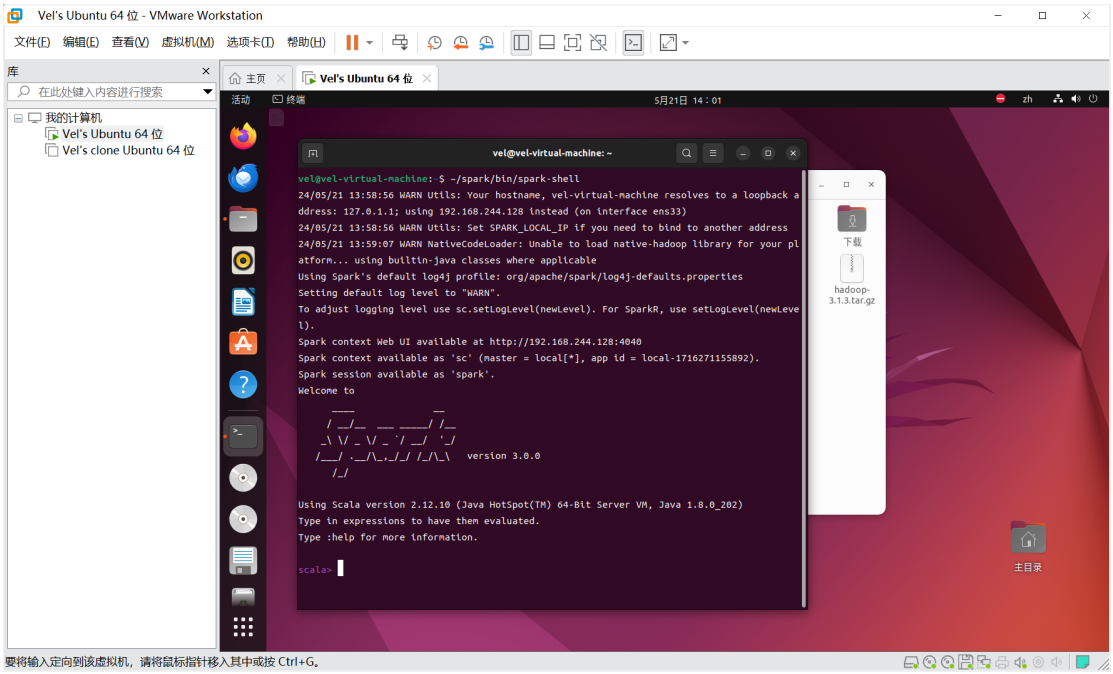
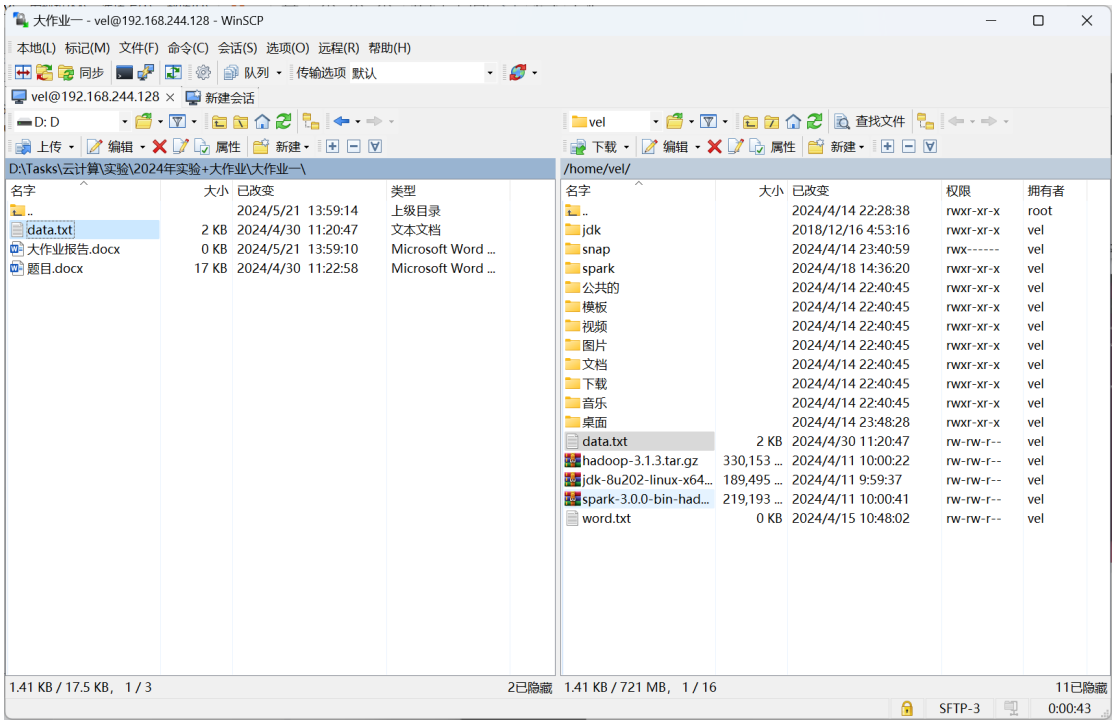
1. 先打开 Spark

~/spark/bin/spark-shell



2. 用 WinSCP 把文件导入



3. 导入库 读取数据

```
import org.apache.spark.sql.SparkSession

import org.apache.spark.sql.functions._


// 读取数据

val filePath = "/home/vel/mnt/data.txt"


// 加载数据并解析每行记录

val data = sc.textFile(filePath)

val parsedData = data.map(line => {

  val fields = line.split(",")

  val userId = fields(0).toInt

  val friendIds = fields.slice(1, fields.length - 3).map(_.toInt)

  val gender = fields(fields.length - 3)

  val age = fields(fields.length - 2).toInt

  val location = fields(fields.length - 1)

  (userId, friendIds, gender, age, location)

})
```

```
scala> // 加载数据并解析每行记录

scala> val data = sc.textFile(filePath)
data: org.apache.spark.rdd.RDD[String] = /home/vel/mnt/data.txt MapPartitionsRDD
[1] at textFile at <console>:33

scala> val parsedData = data.map(line => {
     |   val fields = line.split(",")
     |   val userId = fields(0).toInt
     |   val friendIds = fields.slice(1, fields.length - 3).map(_.toInt)
     |   val gender = fields(fields.length - 3)
     |   val age = fields(fields.length - 2).toInt
     |   val location = fields(fields.length - 1)
     |   (userId, friendIds, gender, age, location)
     | })
parsedData: org.apache.spark.rdd.RDD[(Int, Array[Int], String, Int, String)] = M
apPartitionsRDD[2] at map at <console>:32
```

4. 找出拥有最多好友的前 N 个用户及其好友数量(例如前 10 名)

// 1

// 计算每个用户的好友数量

val friendCounts = parsedData.map{ case (userId, friendIds, _, _, _) => (userId, friendIds.length) }

// 找出前 N 个用户

val topNUsers = friendCounts.takeOrdered(10)(Ordering[Int].reverse.on(_._2))

// 输出结果

topNUsers.foreach(println)

```
scala> // 计算每个用户的好友数量

scala> val friendCounts = parsedData.map{ case (userId, friendIds, _, _, _) => (userId, friendIds.length) }
friendCounts: org.apache.spark.rdd.RDD[(Int, Int)] = MapPartitionsRDD[3] at map at <console>:32

scala>

scala> // 找出前N个用户

scala> val topNUsers = friendCounts.takeOrdered(10)(Ordering[Int].reverse.on(_._2))
topNUsers: Array[(Int, Int)] = Array((1,6), (3,5), (4,4), (2,4), (5,4), (53,3), (6,3), (34,3), (8,3), (11,3))

scala>

scala> // 输出结果

scala> topNUsers.foreach(println)
(1,6)
(3,5)
(4,4)
(2,4)
(5,4)
(53,3)
(6,3)
(34,3)
(8,3)
(11,3)
```

已经找到了拥有最多好友的十个用户，可以得知有人最多拥有十个好友

5. 按照用户年龄段(例如 18-25,26-35,36-45...)统计每个年龄段的平均好友数量

```
// 2

// 定义年龄段函数

def ageGroup(age: Int): String = {

  if (age >= 18 && age <= 25) "18-25"

  else if (age >= 26 && age <= 35) "26-35"

  else if (age >= 36 && age <= 45) "36-45"

  else "Above 45"

}

// 计算每个用户的年龄段及其好友数量
```

val ageFriendCounts = parsedData.map{ case (_, friendIds, _, age, _) => (ageGroup(age), friendIds.length) }

// 按年龄段分组并计算平均好友数量

val avgFriendCountsByAge = ageFriendCounts.groupByKey().mapValues(friendCounts => friendCounts.sum.toDouble / friendCounts.size)

// 输出结果

avgFriendCountsByAge.collect().foreach(println)

```
scala> // 定义年龄段函数

scala> def ageGroup(age: Int): String = {
     |   if (age >= 18 && age <= 25) "18-25"
     |   else if (age >= 26 && age <= 35) "26-35"
     |   else if (age >= 36 && age <= 45) "36-45"
     |   else "Above 45"
     | }
ageGroup: (age: Int)String

scala>

scala> // 计算每个用户的年龄段及其好友数量

scala> val ageFriendCounts = parsedData.map{ case (_, friendIds, _, age, _) => (ageGroup(age), friendIds.lengt
h) }
ageFriendCounts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[5] at map at <console>:34

scala>

scala> // 按年龄段分组并计算平均好友数量

scala> val avgFriendCountsByAge = ageFriendCounts.groupByKey().mapValues(friendCounts => friendCounts.sum.toDo
uble / friendCounts.size)
avgFriendCountsByAge: org.apache.spark.rdd.RDD[(String, Double)] = MapPartitionsRDD[7] at mapValues at <consol
e>:34

scala>
```

```
scala> // 输出结果

scala> avgFriendCountsByAge.collect().foreach(println)
(Above 45,2.0)
(36-45,2.357142857142857)
(18-25,2.6)
(26-35,2.4285714285714284)
```

可以得知年龄 18-25 的年龄段的人平均好友最多

6. 统计不同性别用户的平均好友数量

// 3

// 计算每个用户的性别及其好友数量

val genderFriendCounts = parsedData.map{ case (_, friendIds, gender, _, _) => (gender, friendIds.length) }

// 按性别分组并计算平均好友数量

val avgFriendCountsByGender = genderFriendCounts.groupByKey().mapValues(friendCounts => friendCounts.sum.toDouble / friendCounts.size)

// 输出结果

avgFriendCountsByGender.collect().foreach(println)

```
scala> // 计算每个用户的性别及其好友数量

scala> val genderFriendCounts = parsedData.map{ case (_, friendIds, gender, _, _) => (gender, friendIds.length
) }
genderFriendCounts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[8] at map at <console>:32

scala>

scala> // 按性别分组并计算平均好友数量

scala> val avgFriendCountsByGender = genderFriendCounts.groupByKey().mapValues(friendCounts => friendCounts.su
m.toDouble / friendCounts.size)
avgFriendCountsByGender: org.apache.spark.rdd.RDD[(String, Double)] = MapPartitionsRDD[10] at mapValues at <co
nsole>:34

scala>

scala> // 输出结果

scala> avgFriendCountsByGender.collect().foreach(println)
(F,2.391304347826087)
(M,2.3783783783783785)
```

可以得知女的平均好友数比男性多

7. 按地理位置统计每个地区最受欢迎的用户(拥有最多好友的用户)

// 4

// 计算每个用户的地理位置及其好友数量

val locationFriendCounts = parsedData.map{ case (_, friendIds, _, _, location) => (location, friendIds.length) }

// 找出每个地区最受欢迎的用户

val popularUsersByLocation = locationFriendCounts.reduceByKey(math.max)

// 输出结果

popularUsersByLocation.collect().foreach(println)

```
scala> // 计算每个用户的地理位置及其好友数量

scala> val locationFriendCounts = parsedData.map{ case (_, friendIds, _, _, location) => (location, friendIds.
length) }
locationFriendCounts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[11] at map at <console>:32

scala>

scala> // 找出每个地区最受欢迎的用户

scala> val popularUsersByLocation = locationFriendCounts.reduceByKey(math.max)
popularUsersByLocation: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[12] at reduceByKey at <console>:
32

scala>

scala> // 输出结果

scala> popularUsersByLocation.collect().foreach(println)
(Chengdu,3)
(Beijing,6)
(Guangzhou,4)
(Shanghai,3)
(Shenzhen,2)
(Shanghai ,4)
```

最后已经找到了每个地区最受欢迎的用户